

1. Import All Libraries:

```
In [28]: import requests #for fetching data from API
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score, mean_squared_error
from datetime import datetime, timedelta
import pytz
```

```
In [48]: #OpenWeather API KEY
API_KEY = '7d95bae8446e44197407f72423e6b25e'
BASE_URL = 'https://api.openweathermap.org/data/2.5/'
#Base URL for making API Requests
```

FUNCTIONS

1. Fetch Current Weather Data:

```
In [50]: def get_current_weather(city):

    url = f"{BASE_URL}weather?q={city}&appid={API_KEY}&units=metric"
    response = requests.get(url)    #send the get request to API
    data = response.json()

    return {
        'city' : data['name'],
        'country' : data['sys']['country'],
        'current_temp' : round(data['main']['temp']),
        'feels_like' : round(data['main']['feels_like']),
        'temp_min' : round(data['main']['temp_min']),
        'temp_max' : round(data['main']['temp_max']),
        'pressure' : round(data['main']['pressure']),
        'humidity' : round(data['main']['humidity']),
        'wind_gust_dir' : data['wind']['deg'],
        'WindGustSpeed' : round(data['wind']['speed']),
        'description' : data['weather'][0]['description']
    }
```

```
In [51]: def current_weather(city):
          url = f"{BASE_URL}weather?q={city}&appid={API_KEY}&units=metric"

          print(url)

          response = requests.get(url)
          print(response.status_code)
          print(response.text)

          current_weather('Patna')

https://api.openweathermap.org/data/2.5/weather?q=Patna&appid=7d95bae8446e44197407f72423e6b25e&units=metric
200
{"coord":{"lon":85.1167,"lat":25.6},"weather":[{"id":803,"main":"Clouds","description":"broken
clouds","icon":"04n"}],"base":"stations","main":
{"temp":32.55,"feels_like":37.74,"temp_min":32.55,"temp_max":32.55,"pressure":998,"humidity":58,"sea_level":998,
"grnd_level":992},"visibility":1000,"wind":{"speed":3.49,"deg":98,"gust":7.23},"clouds":
{"all":84},"dt":1781039744,"sys":
{"country":"IN","sunrise":1781047681,"sunset":1781097001},"timezone":19800,"id":1260086,"name":"Patna","cod":200
}
```

2. Read Historical Data:

```
In [14]: def read_hist_data(filename):

          df = pd.read_csv(filename)
          df.dropna() #remove records with missing values
          df.drop_duplicates() #remove records with duplicate data
          return df
```

3. Prepare data for training:

```
In [15]: def prep_data(data):

    le = LabelEncoder()

    #transform categorical column into numerical....
    data['WindGustDir'] = le.fit_transform(data['WindGustDir'])
    data['RainTomorrow'] = le.fit_transform(data['RainTomorrow'])

    #define feature variables & target variables
    X = data[['MinTemp', 'MaxTemp', 'WindGustDir', 'WindGustSpeed',
              'Humidity', 'Pressure', 'Temp']]
    y = data['RainTomorrow'] #target label

    return X, y, le
```

4. Train Rain Tomorrow Prediction Model:

```
In [58]: def train_rain_model(X, y):  
  
    #Train-Test Split  
    X_train, X_test, y_train, y_test = train_test_split(X, y,  
                                                         test_size=0.2, random_state=42)  
  
    #Random Forest Classifier Model  
    model = RandomForestClassifier(n_estimators=100, random_state=42)  
    model.fit(X_train, y_train)  
  
    y_pred = model.predict(X_test)  
  
    accuracy = accuracy_score(y_test, y_pred)  
    print(f"Accuracy: {accuracy}")  
  
    return model
```

5. Prepare Regression Data:

```
In [17]: def prep_reg_data(data, feature):  
  
    X, y = [], [] #initialize list for features & labels  
  
    for i in range(len(data) - 1):  
        X.append(data[feature].iloc[i])  
        y.append(data[feature].iloc[i+1])  
  
    X = np.array(X).reshape(-1, 1)  
    y = np.array(y)  
  
    return X, y
```

6. Train Regression Model:

```
In [18]: def train_reg_model(X, y):  
  
    model = RandomForestRegressor(n_estimators=100, random_state=42)  
    model.fit(X, y)  
  
    return model
```

7. Predict Future:

```
In [57]: def predict_future(model, current_val):
           predictions = [current_val]

           for i in range(5):
               nxt_val = model.predict(np.array([[predictions[-1]]]))
               predictions.append(nxt_val[0])

           return predictions[1:]
```

8. Weather Analysis:

```
In [64]: def weather_view():

    city = input('Enter City: ')
    current_weather = get_current_weather(city)

    #load historical data
    hist_data = read_hist_data('/content/weather.csv')

    #prepare data & train model
    X, y, le = prep_data(hist_data)
    rain_model = train_rain_model(X, y)

    # Map wind direction to compass points
    wind_deg = current_weather['wind_gust_dir'] % 360
    compass_points = [
        ("N", 0, 11.25), ("NNE", 11.25, 33.75), ("NE", 33.75, 56.25),
        ("ENE", 56.25, 78.75), ("E", 78.75, 101.25), ("ESE", 101.25, 123.75),
        ("SE", 123.75, 146.25), ("SSE", 146.25, 168.75), ("S", 168.75, 191.25),
        ("SSW", 191.25, 213.75), ("SW", 213.75, 236.25), ("WSW", 236.25, 258.75),
        ("W", 258.75, 281.25), ("WNW", 281.25, 303.75), ("NW", 303.75, 326.25),
        ("NNW", 326.25, 348.75)]
    compass_direction = next(point for point, start,
                             end in compass_points if start <= wind_deg < end)

    compass_direction_encoded = le.transform(
        [compass_direction])[0] if compass_direction in le.classes_ else -1

    #prepare current weather data
    current_data = {
        'MinTemp': current_weather['temp_min'],
        'MaxTemp': current_weather['temp_max'],
        'WindGustDir': compass_direction_encoded,
        'WindGustSpeed': current_weather['WindGustSpeed'],
```



```
'Humidity': current_weather['humidity'],
'Pressure': current_weather['pressure'],
'Temp': current_weather['current_temp']}]

current_df = pd.DataFrame([current_data])

#rain prediction
rain_prediction = rain_model.predict(current_df)[0]

#prepare regression model for temperature and humidity
X_temp, y_temp = prep_reg_data(hist_data, 'Temp')
X_hum, y_hum = prep_reg_data(hist_data, 'Humidity')

temp_model = train_reg_model(X_temp, y_temp)
hum_model = train_reg_model(X_hum, y_hum)

# predict future temperature & humidity
future_temp = predict_future(temp_model, current_data['Temp'])
future_hum = predict_future(hum_model, current_data['Humidity'])

#prepare time for future predictions
timezone = pytz.timezone('Asia/Kolkata')
now = datetime.now(timezone)
next_hour = now + timedelta(hours=1)
next_hour = next_hour.replace(minute=0, second=0, microsecond=0)

future_times = [(next_hour + timedelta(hours=i)).strftime("%H:00") for i in range(5)]

#Display results

print(f"City: {city}, {current_weather['country']}")
print(f"Current Temperature: {current_weather['current_temp']}")
print(f"Feels Like: {current_weather['feels_like']}")
print(f"Minimum Temperature: {current_weather['temp_min']}°C")
print(f"Maximum Temperature: {current_weather['temp_max']}°C")
print(f"Humidity: {current_weather['humidity']}%")
print(f"Weather Prediction: {current_weather['description']}")
print(f"Rain Prediction: {'Yes' if rain_prediction else 'No'}")
```

```
print("\nFuture Temperature Predictions:")

for time, temp in zip(future_times, future_temp):
    print(f"{time}: {round(temp, 1)}°C")

print("\nFuture Humidity Predictions:")

for time, humidity in zip(future_times, future_hum):
    print(f"{time}: {round(humidity, 1)}%")
```

In [65]: weather_view()

Enter City: Tripura
Accuracy: 0.8378378378378378
City: Tripura, IN
Current Temperature: 24
Feels Like: 26
Minimum Temperature: 24°C
Maximum Temperature: 24°C
Humidity: 99%
Weather Prediction: broken clouds
Rain Prediction: Yes

Future Temperature Predictions:

03:00: 25.7°C
04:00: 22.2°C
05:00: 22.7°C
06:00: 25.0°C
07:00: 26.9°C

Future Humidity Predictions:

03:00: 67.8%
04:00: 72.4%
05:00: 62.3%
06:00: 56.4%
07:00: 45.1%